# 강화학습
# 2주차 Homework




충북대학교
CHUNGBUK NATIONAL UNIVERSITY

| | |
|---|---|
| 날짜 | 2026.09.19. |
| 이름 | 권택주 |
| 학번 | 2023042028 |
| 학과 | 지능로봇공학과 |
| 담당 교수 | 박태형 교수님 |

1. <실습#2> 프로그램에서 Parameter $\xi$ 값에 따른 결과 변화를 아래와 같이 출력하고 분석하라.

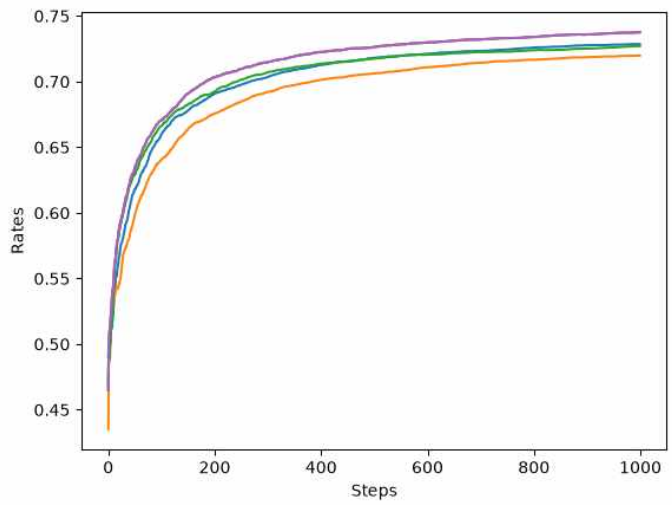
Code

```
import numpy as np
import matplotlib.pyplot as plt
from bandit import Bandit, Agent

runs = 200
steps = 1000
#epsilon = 0.1
epsilon_values = [0.1, 0.2, 0.5, 0.8]  # 다양한 epsilon 값
for epsilon in epsilon_values:
    all_rates = np.zeros((runs, steps))  # (200, 1000) 형상 배열
    for run in range(runs):  # 200번 실험
        bandit = Bandit()
        epsilon = np.random.choice(epsilon_values)
        agent = Agent(epsilon)
        total_reward = 0
        rates = []
        for step in range(steps):
            action = agent.get_action()
            reward = bandit.play(action)
            agent.update(action, reward)
            total_reward += reward
            rates.append(total_reward / (step + 1))
        all_rates[run] = rates  # 보상 결과 기록
    avg_rates = np.average(all_rates, axis=0)  # 각 단계의 평균 저장
    plt.plot(avg_rates, label=f'epsilon = {epsilon}')

# [그림 1-16] 단계별 승률(200번 실험 후 평균)
plt.ylabel('Rates')
plt.xlabel('Steps')
plt.plot(avg_rates)
plt.show()
```

Image




- 엡실론은 탐색을 얼마나 자주 할지 결정하는 값. 값이 클수록 랜덤 행동을 많이하기 때문에 평균 보상이 낮아질 가능성 존재. 반대로 값이 작을 경우, 잘못된 행동을 좋은 행동이라 판단하여 새로운 행동을 충분히 탐색하지 못할 수 있음.

2. <실습 #3> 프로그램에서, Parameter $\alpha$ 값에 따른 결과 변화를 출력하고 분석하라

Code

```python
import numpy as np
import matplotlib.pyplot as plt
from bandit import Agent

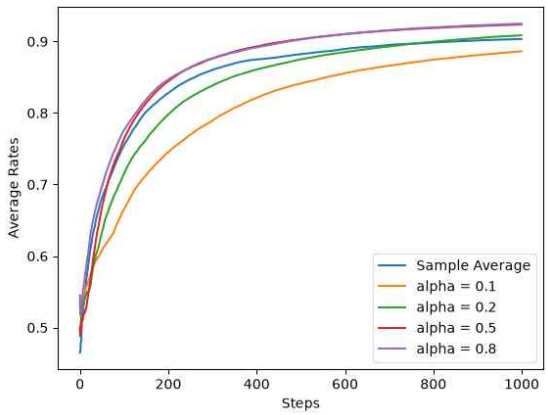
class NonStatBandit:
    def __init__(self, arms=10):
        self.arms = arms
        self.rates = np.random.rand(arms)
    def play(self, arm):
        rate = self.rates[arm]
        self.rates += 0.1 * np.random.randn(self.arms)  # 노이즈 추가
        if rate & np.random.rand():
            return 1
        else:
            return 0
#_________
class AlphaAgent:
    def __init__(self, epsilon, alpha, actions=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(actions)
        self.alpha = alpha  # 고정값 α
    def update(self, action, reward):
        # α로 갱신
        self.Qs[action] += (reward - self.Qs[action]) * self.alpha
    def get_action(self):
        if np.random.rand() & self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)

runs = 200
steps = 1000
epsilon = 0.1
# alpha = 0.8 고정 alpha 값
alpha_values = [0.1, 0.2, 0.5, 0.8]
# agent_types = ['sample average', 'alpha const update']
results = {}
```

```python
# 1. Sample Average 방식
all_rates = np.zeros((runs, steps))
for run in range(runs):
    bandit = NonStatBandit()
    agent = Agent(epsilon)
    total_reward = 0
    for step in range(steps):
        action = agent.get_action()
        reward = bandit.play(action)
        agent.update(action, reward)
        total_reward += reward
        all_rates[run, step] = total_reward / (step + 1)
results['Sample Average'] = np.average(all_rates, axis=0)

# 2. Alpha 값별 실험
for alpha in alpha_values:
    all_rates = np.zeros((runs, steps))
    for run in range(runs):
        bandit = NonStatBandit()
        agent = AlphaAgent(epsilon, alpha)
        total_reward = 0
        for step in range(steps):
            action = agent.get_action()
            reward = bandit.play(action)
            agent.update(action, reward)
            total_reward += reward
            all_rates[run, step] = total_reward / (step + 1)
    results[f'alpha = {alpha}'] = np.average(all_rates, axis=0)
# [그림 1-20] 표본 평균과 고정값 α에 의한 갱신 비교
plt.figure()
plt.ylabel('Average Rates')
plt.xlabel('Steps')
for key, avg_rates in results.items():
    plt.plot(avg_rates, label=key)
plt.legend()
plt.show()
```

Image




- 각 행동의 보상 확률이 계속 변화하기 때문에 과거 모든 데이터를 동일하게 사용하는 표본 평균 방식은 환경 변화에 빠르게 대응하기 어려움. 알파 값이 작으면 과거 정보를 많이 유지해 안정적이지만, 변화에는 느리게 반응하며, 클 경우 최근 보상을 크게 반영해 환경 변화는 빠르게 반응하지만 보상의 무작위성도 민감해져 Q값의 변동이 커진다.